

constexpr Function Parameters

Document Number: P1045R0

Date: 2018-04-29

Author: David Stone (davidmstone@google.com)

Audience: Evolution Working Group (EWG)

Abstract

With this proposal, the following code would be valid:

```
void f constexpr (int x) {  
    static_assert(x == 5);  
}
```

The parameter is usable in all the same ways as any constexpr variable.

Moreover, this paper proposes the introduction of a "maybe constexpr" qualifier, with a proposed syntax of constexpr? (this syntax should be seen as a placeholder if a better syntax is suggested). Such a function can accept values that are or are not constexpr and maintain that status when passed on to another function. In other words, this is a way to deduce and forward constexpr, similar to what "forwarding references" / "universal references" (T &&) do in function templates today. This paper proposes adding generally usable functionality to test whether an expression is a constant expression (is_constexpr), with the primary use case being to use this test in an if constexpr branch in such a function to add in a compile-time-only check.

Finally, this paper proposes allowing overloading on constexpr parameters. Whether a parameter is constexpr would be used as the final step in function overload resolution to resolve cases that would otherwise be ambiguous. Put another way, we first perform overload resolution on type, then on constexpr.

This paper has three primary design goals:

1. Eliminate arcane metaprogramming from modern libraries by allowing them to make use of "regular" programming.
2. Allow library authors to take advantage of known-at-compile-time values to improve the performance of their libraries
3. Allow library authors to take advantage of known-at-compile-time values to improve the diagnostics of their libraries

No Shadow Worlds

Malte Skarupke wrote an article on [language design and "shadow worlds"](#). Shadow worlds are parts of a language which are segregated from the rest of the language. When you are stuck in this "shadow" language, you find yourself wanting to use the full power of the "real" language. C++ contains at least four general purpose languages (three of them shadow languages): "regular" C++, constexpr functions, templates, and macros.

Prior to C++14, constexpr functions couldn't have if or for, leading to contorted recursive code full of return condition ? complicated_expression1 : complicated_expression2. Since C++14, we cannot allocate memory in constexpr functions. Once we remove that restriction in C++20, we still cannot reinterpret_cast. The trend here is an ever increasing move toward making constexpr functions just be regular functions.

Templates are in an even worse spot: another Turing complete language that has no loops, no mutation, and a completely different syntax for everything.

Worse still are macros: no loops, mutation, [recursion](#), scoping, no real functions, and weird if statements.

For this paper, the shadow world of templates is the most relevant. In a strong twist of irony, Malte's article talks about macros being a shadow world in every language, so therefore you want a strong template system that can replace macros. The template system, however, is yet another shadow language. In C++, as in other languages, the compile-time arguments are split from the run-time: we have template parameters and function parameters, so you can write f<0>(1). That seems fine, at first, until you realize that there are many things you cannot do because of that syntactic difference.

You cannot pass template arguments to constructors. You cannot pass template arguments to overloaded operators. You cannot overload on whether a value is known at compile time when they have different syntaxes. You cannot put a compile-time argument after a run-time argument, even if that is where it makes the most sense for it to be. You cannot generically forward a

pack of arguments, some of them compile time and some of them run time. All of these limitations lead to us reimplementing basic functionality like loops, partial function application, and sorting for "run time" vs "compile time" values.

These problems motivate this paper. By treating compile-time and run-time values more uniformly, we bring these two worlds together. Large portions of existing metaprogramming libraries going back to Boost.MPL try to work around the problems, but we need a language change to solve it at the root.

Before and After

Now (without this proposal)	The future (with this proposal)
<pre>auto a = std::array<int, 2>{}; a[0] = 1; a[1] = 5; a[2] = 3; // undefined behavior auto t = std::tuple<int, std::string>{}; std::get<0>(t) = 1; std::get<1>(t) = "asdf"; std::get<2>(t) = 3; // compile error</pre>	<pre>auto a = std::array<int, 2>{}; a[0] = 1; a[1] = 5; a[2] = 3; // compile failure auto t = std::tuple<int, std::string>{}; t[0] = 1; t[1] = "asdf"; t[2] = 3; // compile failure</pre>
<code>std::true_type{}</code>	<code>true</code>
<pre>std::integral_constant<int, 24>{} std::constant<int, 24> // proposed boost::hana::int_c<24> boost::mpl::int_<24></pre>	<code>24</code>
<pre>template<int n> void f(boost::hana::int_<n>);</pre>	<code>f(constexpr int x);</code>
<pre>0 <==> 1 == 0; // valid, as intended 0 <==> 1 == nullptr; // valid, due to implementation detail</pre>	<pre>0 <==> 1 == 0; // valid, as intended 0 <==> 1 == nullptr; // error: no overloaded operator== comparing // strong_equality with nullptr_t</pre>
<pre>// Either all regex constructions have an extra // pass over the input to determine the parsing // strategy, or... auto const pathological = std::regex("(A+ B+)*C"); // exponential time against failed matches // starting with 'A' and 'B'. // See https://swtch.com/~rsc/regexp/regexp1.html</pre>	<pre>// Scan can be done at compile time, so... auto const pathological = std::regex("(A+ B+)*C"); // linear time against failed matches // starting with 'A' and 'B'.</pre>
<pre>auto const glob = std::regex(""); // throws std::regex_error</pre>	<pre>auto const glob = std::regex(""); // static_assert failed "Regular expression token // '*' must occur after a character to repeat."</pre>
<code>// ???</code>	<pre>static_assert(pow(2_meters, 3) == 8_cubic_meters); meters runtime_value; std::cin >> runtime_value; if (pow(runtime_value, 2) > 100_square_meters) { throw std::exception{}; }</pre>
<pre>// Do I want my code to work at compile time, do I // want my code to be fast, or do I try to modify // my compiler to recognize the code and generate // the right assembly at run time when it write it // in the `constexpr` style?</pre>	<pre>// From the std-proposals forum std::size_t strlen(constexpr char const * s) { for (const char *p = s; ; ++p) { if (*p == '\0') { return static_cast<std::size_t>(p - s); } } } std::size_t strlen(char const * s) { __asm__("SSE 4.2 insanity"); }</pre>
<pre>// Cannot write a function that transparently uses // this intrinsic where possible. Cannot write a // function that puts the parameters in the</pre>	<pre>#include <emmintrin.h> void f(int something, char y) { __m128i x;</pre>

```
// correct order without resorting to  
// `integral_constant` business.
```

```
// ...  
// This intrinsic requires the second argument  
// to be a compile-time constant  
x = __builtin_ia32_aeskeygenassist128(x, y);  
// ...  
}
```

Background

This proposal will reference a few libraries that make heavy use of compile-time parameters. If readers are not familiar with these libraries, that is fine, the below should be sufficient description to understand some of the motivation of this paper. This paper will also reference other papers that are currently under discussion.

boost::mpl

The [Boost.MPL](#) library is a general-purpose template metaprogramming framework of compile-time algorithms, sequences and metafunctions. It was designed in the constraints of C++03 and organizes itself around passing types to class template parameters.

boost::hana

[Boost.Hana](#) is a metaprogramming library that aims to make metaprogramming just "programming", using as much "regular" C++ as possible. It does this by merging types and values -- values are encoded in types, and types are turned into values -- so that most of the user interaction with `boost::hana` is by calling what appear to be regular functions.

bounded::integer

[bounded::integer](#) is a library that adds compile-time range checking to integer types. `bounded::integer<5, 1000>` is an integer type that can be between 5 and 1000, inclusive, and the resulting type of `bounded::integer<1, 10> + bounded::integer<2, 5>` is `bounded::integer<3, 15>`.

Class Types in Non-Type Template Parameters

[P0732R1](#): current proposal making its way through the committee. From the paper: "We should allow non-union class types to appear in non-type template parameters. Require that types used as such, and all of their bases and non-static data members recursively, have a non-user-provided operator<=> returning a type that is implicitly convertible to `std::strong_equality`, and contain no references." This paper will refer to such types as having a "trivial operator<=>".

Why we need this proposal

`constexpr` parameters have several advantages over existing solutions:

- Better user experience due to uniform syntax on the caller's side: They do not need to remember `f(true_)` or `f(constant<true>)` for `f<true>()` or anything else like that. The standard and most intuitive way to pass a value to a function is in the function parameter list: `f(true)`.
- Better compile times as a result of being able to just pass things like `true` rather than instantiating a wrapper template. This applies primarily to solutions like `boost::mpl` (where everything has to be a type, including integers, and is the origin of `integral_constant`) and `boost::hana` (where everything is passed as a regular function parameter, even compile-time constants, which uses the equivalent of `integral_constant` but passed by value instead of by type). The cost of time and memory to compile `void f(constexpr int x)` should be essentially the same as `template<int x> void f()`.
- Library authors can add compile-time diagnostics without adding run-time code generation for checking. This is somewhat possible now, but it requires that the caller put the result in a `constexpr` context and does not work in all cases (for instance, `std::regex` cannot yet be a literal type due to possible memory allocation). There is a large amount of user confusion today when a `constexpr` function is passed all `constexpr` parameters, but an error is caught at run time instead of compile time because the result of that function was not stored in a `constexpr` result.
- Allows adding a new overload that can take advantage of known-at-compile-time values, without forcing all callers to change
- Allows easier integration of strongly-typed code with existing code. There are many generic libraries that expect to be able to initialize integer types with a literal `0` or `1`. To support this, the `bounded::integer` library would have to accept all arguments of type `int` (and hope that the library never passes `0U`), which removes some of the compile-time safety from the

library. If `bounded::integer` could know that it was dealing with a safe value, it would go a long way toward improving interoperability.

Details

In which contexts can constexpr parameters be used?

This paper proposes allowing a `constexpr` parameter anywhere that any other `constexpr` value could be used in any location that appears lexically after the parameter. In this way, a `constexpr` parameter is usable in much the same way as a template parameter. In particular, the following code is valid:

```
void f constexpr (int X, std::array<int, X> const & a)
    noexcept(X > 5)
    requires X % 2 == 0
{
    static_assert(X == 6);
}
```

How do function static variables interact with this feature?

The mental model users should have for this feature is that of a regular function for which some parameters are available for use at compile time. It may also be tempting to think of this purely as an alternative syntax for function templates, and mentally imagining a "desugaring" to a function template. Those two models are not in conflict for most cases, but when static variables come into play, they suggest very different results.

```
int & f constexpr (int) {
    static int x = 0;
    return x;
}
++f(0);
std::cout << f(1);
```

What is this program guaranteed to print? The "secretly a template" model says it should print "0" because these are two different functions. The "regular function" model says it should print "1" because the variable `x` is declared static, and since we have written just one function here, we are using the same variable. This paper strongly recommends against the "secretly a template" model, since one of primary goals of this proposal is to allow a more regular style of programming. This puts slightly more of a burden on implementors because it means that they can reuse slightly less of the existing template machinery, but it should not be too much extra work to make all of the static variables reference the same variable [[TODO: citation needed]].

What are the restrictions on types that can be used as constexpr parameters?

There are two possible options here. Conceptually, we would like to be able to use any literal type as a `constexpr` parameter. This gives us the most flexibility and feels most like a complete feature. Again, however, the obvious implementation strategy for this paper would be to reuse the machinery that handles templates now: internally treat `void f constexpr (int x)` much like `template<int x> void f()`. In the "template" model, we would limit `constexpr` parameters to be types which can be passed as template parameters: with P0732, types with a trivial operator`<=>`. For functions with `constexpr` parameters, however, many of the concerns that prompted that requirement go away under the "regular function" model. The importance of "same instantiation" is limited thanks to not trying to conceptually "instantiate" this into a "regular" function. Because we do not propose allowing the formation of pointers to such functions and because we propose a single instance of static variables, the problem is avoided entirely. We do not need to decide whether two functions with different `constexpr` variables are "the same" function because the answer is always yes for all types of parameters: `f(1)` calls the same function as `f(2)` even if the parameter is `constexpr`.

Can you take the address of constexpr parameter?

Just like in P0732, we would like to be able to call const member functions on these parameters at run time, but for that to be possible they need to have an address. To satisfy that requirement, `constexpr` parameters should be const objects, of which there is one instance in the program for every distinct `constexpr` parameter value. P0732 proposes this only for non-type template parameters of class type, but there seems to be no reason to limit this in that way. For consistency, it seems that this should be allowed for all `constexpr` parameters, not just those of class type, and the same argument applies to all non-type template parameters.

Function Pointers

This paper does not propose adding support for pointers to functions accepting constexpr parameters simply because the full interactions there have not yet been explored. Theoretically this could be supported if the type of the function pointer was `void(constexpr int)` and the function pointer is formed in a constexpr context, but that last requirement would be a completely new type of restriction.

Perfect forwarding

It would be nice to be able to write a function template that accepts parameters that might be constexpr and forward them on to another function. For example, if I have a class that wraps another type, it would be nice to write a constructor that forwards all arguments on to the wrapped type. To do this, we need some sort of perfect forwarding mechanism. There are three possibilities here:

1. Do not solve this problem. Ignoring this problem causes an exponential explosion on forwarding functions and makes it impossible to write variadic versions of such functions.
2. Follow the lead of rvalue vs. lvalue references and implement a "constexpr deduction" rule. In other words, for `template<typename T> void f(T x)`, the type `T` can be deduced as `constexpr int`. This seems appealing at first, because of the analogy to rvalue references and the fact that existing code would automatically become "aware" of this feature and take advantage of it. However, that is both a benefit and a drawback, because it means that all function templates get this behavior. This could have a very large cost in time and memory at compile time if compilers have to activate much of their current template machinery to implement this. This also requires using a deduced type even when the exact type is known.
3. "Maybe constexpr" parameters. `void f(constexpr? int x)` is a (strawman) syntax to allow users to write a single function overload that can accept parameters that can be used as compile-time constants and parameters which cannot.

Option 3 seems like the best option, but requires some more explanation for how it would work. There are two primary use cases for this functionality. The first use case is to forward an argument to another function that has a constexpr? parameter or is overloaded on constexpr. The second use case is to write a single function that has one small bit of code that is run only at compile time or only at run time. For instance, `std::array::operator[]` could be implemented like this:

```
auto & operator[](constexpr? size_t index) {
    if constexpr(IS_CONSTEXPR(index)) {
        static_assert(index < size());
    }
    return __data[index];
}
```

Here, we put in a compile-time check where possible, but otherwise the behavior is the same. I actually expect this to be the primary way to use constexpr parameters when overloading with run-time parameters, as it ensures there are no accidental differences in the definition.

Here, `IS_CONSTEXPR` is defined as a macro, which is possible to write in standard C++ with the introduction of this paper:

```
template<typename T>
std::true_type is_constexpr_impl(constexpr T &&);

template<typename T>
std::false_type is_constexpr_impl(T &&);

#define IS_CONSTEXPR(...) decltype(is_constexpr_impl(__VA_ARGS__)){}
```

This must be a macro so that it does not evaluate the expression and perform side effects. We need to make it unevaluated, and the macro wraps that logic. I believe it shows the strength of this solution that users can use it to portably detect whether an expression is constexpr (a common user request for a feature), rather than needing that as a built-in language feature. Regardless of how the feature is specified it is an essential component of this proposal.

Effect on the standard library

This paper is expected to have a fairly small impact on the standard library, because we currently make use of few non-type template parameters on function templates. The most obvious change is that we should add `operator[]` to `std::tuple`, but I will write a separate paper, pending the approval of this proposal, that addresses library impact.

Effect on other parts of the language

We also have `std::get` baked into the language in the form of structured bindings. Users should be able to opt-in to structured bindings with `std::tuple_size`, `std::tuple_element`, and `operator[]` instead of `std::tuple_size`, `std::tuple_element`, and `get<i>`. Right now, the tuple-like protocol first tries member `get`, then `std::get`. We could add `operator[]` into the mix, either as the first option or the last option.

Related work

Daveed's [constexpr operator](#) allows the user to test whether the current evaluation is in a `constexpr` context, so the user could say `if constexpr (constexpr()) {} else {}`. This solves some of the problems addressed by this paper, but not all. Consider an alternative universe where we want to restrict memory orders to be compile-time constants, rather than the current state where they could be run-time values. With `constexpr` parameters, the solution is straightforward:

```
void atomic<T>::store(T desired, constexpr std::memory_order order = std::memory_order_seq_cst) noexcept;
```

This would require the user to pass the memory order as a compile-time constant, but places no constraints on the overall execution being `constexpr` (and in fact, we know it never will be because `atomic` is not a literal type). It is not possible to implement this with a `constexpr` block. In general, this applies to any situation where you currently have a function accepting a function parameter and a template parameter (or a regular function parameter and a function parameter wrapped in something like `std::integral_constant`). For a perhaps more compelling example, this is how we would have originally designed tuple had we had `constexpr` parameters:

```
constexpr auto && tuple::operator[](constexpr std::size_t index) { ... }
```

where the tuple variable (`*this`) is (potentially) a run time value. In other words, the code using the index is mixed in with the code using the tuple. The `constexpr()` operator allows you to write different code depending on whether the entire evaluation is part of a `constexpr` context. `constexpr` parameters allows you to partially apply certain arguments at compile time without needing to compute the entire expression at compile time.